

RELATÓRIO TÉCNICO

Entrega Contínua de uma Plataforma de Pedidos em Microsserviços
Do Docker Compose ao Kubernetes com Observabilidade e CI/CD

Loja Veloz · Cloud DevOps Engineer · 2025

1. Introdução e Contexto

A Loja Veloz é uma empresa de varejo digital em forte crescimento que enfrenta desafios críticos de confiabilidade e escalabilidade: indisponibilidades durante deploys, dificuldade de escalar em picos de acesso e baixa rastreabilidade de falhas entre serviços. Este relatório documenta a fundamentação teórica que orienta a modernização da plataforma **Pedidos Veloz**, abrangendo arquitetura de microsserviços, containerização, orquestração, CI/CD e observabilidade.

2. Microsserviços e DevOps Cloud-Native

2.1 Arquitetura de Microsserviços

A arquitetura de microsserviços decompõe uma aplicação monolítica em serviços pequenos, coesos e implantáveis de forma independente, cada um responsável por um domínio de negócio específico (Newman, 2015). Os princípios fundamentais são: **alta coesão** (cada serviço faz uma coisa bem), **baixo acoplamento** (comunicação via APIs ou eventos) e **propriedade de dados** (cada serviço possui seu próprio armazenamento).

Na Loja Veloz, a plataforma é composta por: API Gateway (roteamento e autenticação), Serviço de Pedidos, Serviço de Pagamentos, Serviço de Estoque e banco PostgreSQL compartilhado na fase inicial, com evolução prevista para bancos por domínio. A comunicação assíncrona via mensageria (ex.: RabbitMQ ou Kafka) é adotada para eventos como *PedidoCriado*, desacoplando os serviços produtores dos consumidores e aumentando a resiliência.

2.2 O Papel do DevOps em Ambientes Cloud-Native

O DevOps é um conjunto de práticas culturais, organizacionais e técnicas que aproxima desenvolvimento e operações com o objetivo de entregar software com maior frequência, qualidade e estabilidade (Humble & Farley, 2010). Em ambientes cloud-native, o DevOps se apoia em quatro pilares:

- Automação:** pipelines de CI/CD eliminam trabalho manual e reduzem erros humanos.
- Infraestrutura como Código (IaC):** ambientes reproduzíveis via Terraform.
- Observabilidade:** métricas, logs e traces para diagnóstico rápido de falhas.
- Resiliência por design:** health checks, retry, circuit breaker e deploys graduais.

A metodologia **12-Factor App** (Heroku, 2011) complementa o DevOps definindo boas práticas como: configuração via variáveis de ambiente, processos stateless, paridade entre dev/prod, logs como streams e disposability (inicialização rápida e desligamento gracioso). Todos esses princípios são adotados neste projeto.

3. Containerização: Docker vs Kubernetes

3.1 Docker e Docker Compose

O Docker empacota aplicações e suas dependências em **contêineres** — unidades isoladas e portáteis que executam de forma consistente em qualquer ambiente (Docker Inc., 2013). As principais vantagens são: eliminação do problema "funciona na minha máquina", imagens imutáveis e versionadas, e inicialização em segundos. Boas práticas adotadas: builds multi-stage para imagens enxutas, execução com usuário não-root e uso de imagens base mínimas (alpine/distroless).

O **Docker Compose** orquestra múltiplos contêineres localmente com um único arquivo YAML, definindo redes, volumes, variáveis de ambiente e dependências entre serviços. É ideal para o ambiente de desenvolvimento e testes de integração local.

3.2 Kubernetes para Produção

O Kubernetes (K8s) é um orquestrador de contêineres de código aberto que automatiza implantação, escalonamento e gerenciamento de aplicações containerizadas (CNCF, 2014). Diferentemente do Compose, o K8s foi projetado para produção: oferece autodiscovery, self-healing, rolling updates, escalonamento automático e gerenciamento declarativo de configurações.

Critério	Docker Compose	Kubernetes
Ambiente alvo	Desenvolvimento local / CI	Produção / staging
Escalação	Manual (scale)	Automático (HPA/VPA)
Self-healing	Não	Sim (ReplicaSet)
Complexidade	Baixa	Alta (curva de aprendizado)
Caso de uso	Dev/teste padronizado	Cargas de trabalho reais

Tabela 1 – Comparativo Docker Compose vs Kubernetes

4. Fundamentos Teóricos

4.1 Orquestração de Contêineres

A orquestração gerencia o ciclo de vida de contêineres em escala. No Kubernetes, os principais recursos são: **Pod** (menor unidade de implantação), **Deployment** (controle declarativo de réplicas e atualizações), **Service** (abstração de rede estável), **ConfigMap/Secret** (configurações e credenciais separadas do código), **HPA** (Horizontal Pod Autoscaler – escala com base em métricas como CPU/memória) e **Ingress** (roteamento HTTP/HTTPS). Readiness e liveness probes garantem que o tráfego só é direcionado a pods saudáveis e que pods com falha são reiniciados automaticamente.

4.2 CI/CD em Ambientes Distribuídos

Integração Contínua (CI) é a prática de integrar e validar código automaticamente a cada commit, executando build, lint e testes. **Entrega Contínua (CD)** estende o CI publicando artefatos validados em ambientes de staging e produção de forma automatizada e auditável (Humble & Farley, 2010).

Em ambientes de microsserviços, pipelines independentes por serviço permitem releases desacoplados. A estratégia adotada para a Loja Veloz utiliza **GitHub Actions** com os seguintes estágios: (1) lint e

testes unitários, (2) build e push de imagem Docker com tag semântica, (3) scan de vulnerabilidades (Trivy/Grype), (4) deploy em Kubernetes via kubectl/Helm com validação de rollout.

4.3 Observabilidade

Observabilidade é a capacidade de inferir o estado interno de um sistema a partir de suas saídas externas. Os três pilares são (Majors, 2020):

- **Métricas:** dados numéricos agregados no tempo (ex.: latência p95, taxa de erros, uso de CPU). Coletadas via Prometheus e visualizadas no Grafana.
- **Logs:** registros estruturados (JSON) de eventos do sistema. Centralizados via Fluentd/Loki para correlação e busca eficiente.
- **Traces:** rastreamento distribuído que acompanha uma requisição por todos os serviços. Instrumentado com OpenTelemetry e visualizado no Jaeger/Tempo.

O **service mesh** (Istio/Linkerd) complementa a observabilidade adicionando telemetria automática, mTLS entre serviços e controle de tráfego (canary, circuit breaker) sem alterações no código da aplicação.

5. Justificativa das Decisões Arquiteturais

• Estratégia de Deploy: Rolling Update

Atualização gradual das réplicas garante zero downtime e reversão rápida caso a nova versão apresente falhas. A evolução natural é canary releases para validar novas versões com subconjunto do tráfego antes do rollout total.

• Escalabilidade: HPA com métricas customizadas

O HPA escala pods horizontalmente com base em CPU e, futuramente, em métricas de negócio (ex.: tamanho da fila de pedidos). Para picos previsíveis como campanhas promocionais, Pre-scaling manual complementa o HPA.

• Mensageria assíncrona para eventos

O evento PedidoCriado é publicado em um broker (RabbitMQ) para desacoplar Pedidos de Pagamentos e Estoque, garantindo resiliência: se Estoque estiver indisponível, o pedido não é perdido.

• IaC com Terraform

Toda infraestrutura cloud (VPC, cluster GKE/EKS, load balancers, registros) é provisionada via Terraform, garantindo reprodutibilidade, versionamento e auditoria de mudanças via pull requests.

• Segurança por camadas

Secrets gerenciados via Kubernetes Secrets (com integração a Vault em produção), Network Policies restringindo tráfego entre namespaces, imagens escaneadas no CI e Pod Security Admission com perfil 'restricted'.

6. Referências

CNCF. *Kubernetes Documentation*. Disponível em: <https://kubernetes.io/docs>. Acesso em: 2025.

Docker Inc. *Docker Documentation*. Disponível em: <https://docs.docker.com>. Acesso em: 2025.

Heroku. *The Twelve-Factor App*. Disponível em: <https://12factor.net>. Acesso em: 2025.

HUMBLE, J.; FARLEY, D. *Continuous Delivery*. Addison-Wesley, 2010.

NEWMAN, S. *Building Microservices*. O'Reilly Media, 2015.

MAJORS, C. et al. *Observability Engineering*. O'Reilly Media, 2020.

HashiCorp. *Terraform Documentation*. Disponível em: <https://developer.hashicorp.com/terraform>. Acesso em: 2025.

OpenTelemetry. *OpenTelemetry Documentation*. Disponível em: <https://opentelemetry.io/docs>. Acesso em: 2025.

GitHub. *GitHub Actions Documentation*. Disponível em: <https://docs.github.com/actions>. Acesso em: 2025.

Istio. *Istio Documentation*. Disponível em: <https://istio.io/latest/docs>. Acesso em: 2025.